第5章

数组和广义表

本章内容

5.1 数组的类型定义

5.2 数组的顺序表示和实现

5.3 矩阵的压缩存储

5.4 广义表的类型定义

5.5 广义表的存储结构

5.6 广义表的递归算法

§5.1 数组的类型定义

数组和广义表可以看做表中元素本身也是一个数据结构的线性表。

数组中的元素必须属于同一数据类型。

抽象数据类型数组的定义：

ADT Array {

数据对象：

$$D = \{a_{j_1,j_2,\ldots,j_i,j_n} \mid j_i = 0,\ldots,b_i - 1,\ i=1,2,\ldots,n\}$$

数据关系：

$$R = \{R1, R2, \ldots, Rn\}$$

$$Ri = \{<a_{j_1,\ldots j_i,\ldots j_n}, a_{j_1,\ldots j_i+1,\ldots j_n}> \mid 0 \le j_k \le b_k - 1,$$
$$1 \le k \le n \text{ 且 } k \ne i,\ 0 \le j_i \le b_i - 2,\ i=2,\ldots,n\}$$

基本操作：

基本操作：

InitArray(&A, n, bound1, ..., boundn)

DestroyArray(&A)

Value(A, &e, index1, ..., indexn)

Assign(&A, e, index1, ..., indexn)

InitArray(&A, n, bound1, ..., boundn)

操作结果：若维数n 和各维长度合法，则构造相应的数组A，并返回OK。

DestroyArray(&A)

操作结果：销毁数组A。

Value(A, &e, index1, ..., indexn)

初始条件：A是n维数组，e为元素变量，随后是n 个下标值。

操作结果：若各下标不超界，则e赋值为所指定的A 的元素值，并返回OK。

Assign(&A, e, index1, ..., indexn)

初始条件：A是n维数组，e为元素变量，随后是n 个下标值。

操作结果：若下标不超界，则将e的值赋给所指定的A的元素，并返回OK。

二维数组的定义:

数据对象:

$$D = \{a_{ij} \mid 0 \leqslant i \leqslant b_1-1,\ 0 \leqslant j \leqslant b_2-1\}$$

数据关系:

$$R = \{ ROW, COL \}$$

$$ROW = \{<a_{i,j}, a_{i+1,j}> \mid 0 \leqslant i \leqslant b_1-2,\ 0 \leqslant j \leqslant b_2-1\}$$

$$COL = \{<a_{i,j}, a_{i,j+1}> \mid 0 \leqslant i \leqslant b_1-1,\ 0 \leqslant j \leqslant b_2-2\}$$

- 二维数组可以看成是这样一个定长线性表：它的每个数据元素也是一个定长线性表。
- 数组一旦被定义，它的维数和维界就不会再改变，因此，除了结构的初始化和销毁之外，数组只有存取元素和修改元素值的操作。

§5.2 数组的顺序表示和实现

数组特点：

1) 只有引用型操作，没有加工型操作；数组一般不作插入或删除操作，一旦建立了数组，数组中的元素个数和元素之间的关系就不再变化。

   因此，采用顺序存储结构表示数组。

2) 数组是多维的结构，而存储空间是一个一维的结构。

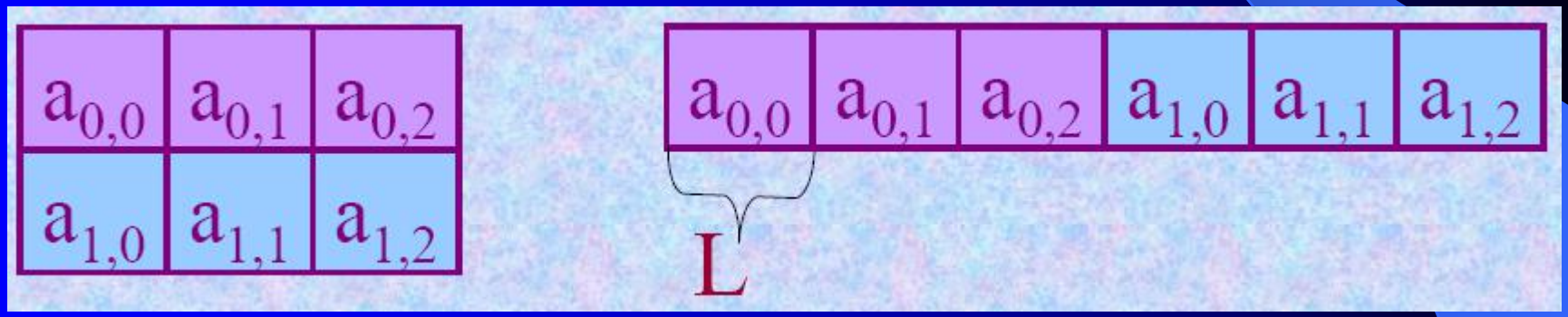
- 以二维数组为例
- 有两种顺序映象的方式：

  1) 以行序为主序(低下标优先)；

  2) 以列序为主序(高下标优先)。
- 对于数组，一旦确定了维数和各维的长度，就可以为它分配存储空间。反之，只要给出一组下标就可以求得相应数组元素的存储位置。

以“行序为主序”的存储映象

例如：



以行序为主序的存储结构

- 设每个元素占L个存储单元，b_1, b_2为数组行长度和列长度，则二维数组A中任一元素a_{ij}的存储位置为：

$$LOC(i, j)=LOC(0, 0)+(b_2\times i+j)L$$

其中，LOC(0, 0)是a_{00}的存储位置，即二维数组A的起始存储位置，也称为基地址或基址。

- 推广到一般情况，n维数组的元素存储位置：

$LOC(j_1,j_2,\ldots,j_n)= LOC(0,0,\ldots,0)$

$+(b_2\times\ldots\times b_n\times j_1+b_3\times\ldots\times b_n\times j_2+\ldots+b_n\times j_{n-1}+j_n)L$

$= LOC(0,0,\ldots,0)+(\Sigma j_i \Pi b_k+j_n)L$

$= LOC(0,0,\ldots,0)+(\Sigma c_i\times j_i)$

$c_n=L$， $c_{i-1}=b_i\times c_i$, $(1<i\leq n)$

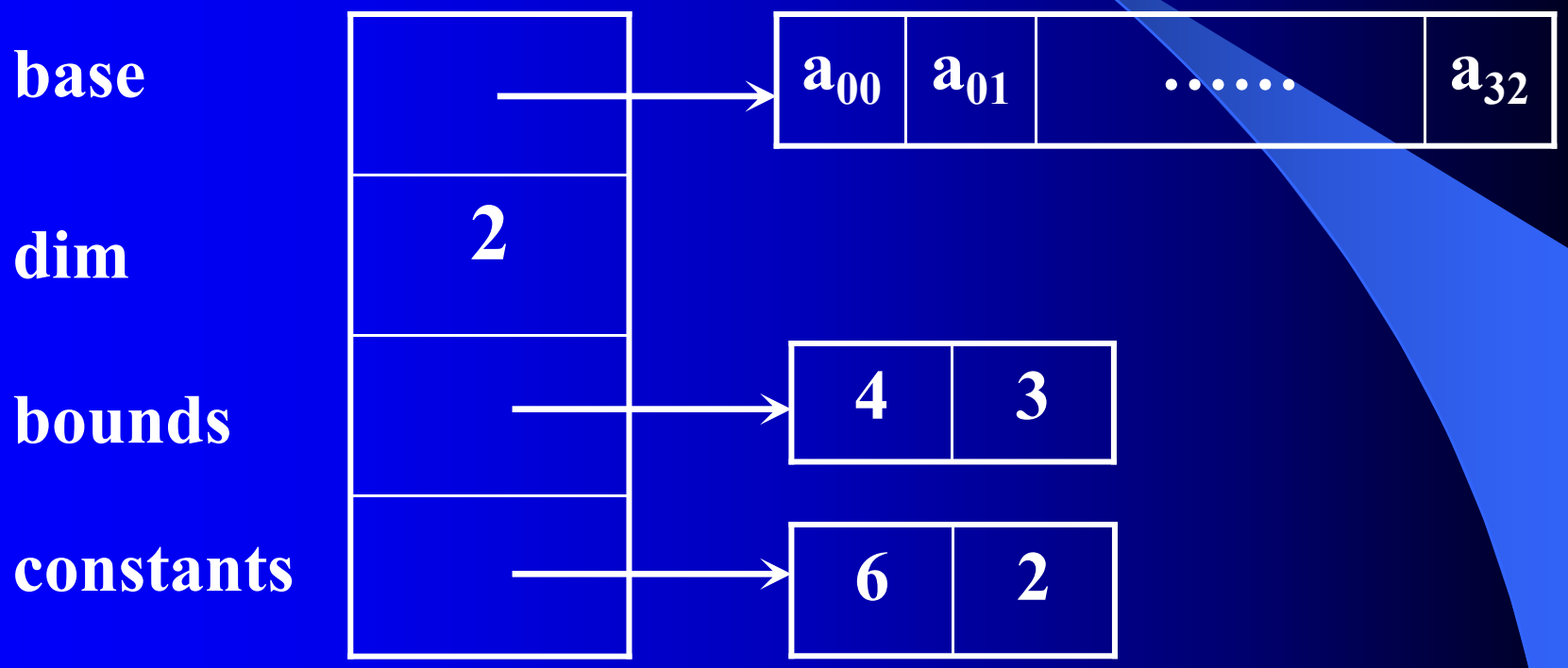
n维数组的映像函数

随机存储结构

数组顺序存储表示和实现

```
define MAX-ARRAY-DIM   8
        //设维数最大值为8
typedef struct {
ElemType  *base ; //数组元素基址
int dim ;            //数组维数
int *bounds ;     //数组维界基址
int *constants ; //数组映象函数常量基址
}  Array ;
```

int A[4][3] 的存储结构



Status InitArray (Array &A, int dim, …)
//若维数dim和随后的各维长度合法，则构造相应的数组A，并返回OK

Status DestroyArray(Array&A)
//销毁数组A

status Value (Array A, ElemType&e, …)
/* A是n维数组，e为元素变量，随后是n个下标值。若各下标不超界，则e赋值为所指定的A的元素值，并返回OK */

status Assign(Array &A,ElemType e,…)
/*A是n维数组，e为元素变量，随后是n个下标值。若下标不超界，则将e的值赋给所指定的A的元素，并返回OK。*/

数组基本操作的算法描述

```
Status InitArray (Array &A, int dim, … )
{  //数组的初始化算法
if (dim<1||dim>MAX-ARRAY-DIM)
                return ERROR;
A. dim=dim;
if (!(A.bounds=(int*)malloc(dim*sizeof(int))))
exit(OVERFLOW);;
//若各维长度合法，则存入A.bounds,并求出A的元素
   总数
elemtotal=1;
va-start(ap,dim);
//ap为va-list类型，是存放变长参数表信息的数组
```

```
for(i=0; i<dim;++i) {
A. bounds[i]=va_arg(ap,int);
if (A.bounds[i]<0)return UNDERFLOW;
elemtotal*=A.bounds[i];}
va_end(ap);
A.base=(ElemType*)malloc(elemtotal*sizeof(ElemTyp
   e));
if(!A.base)exit(OVERFLOW);
if(!(A.constants=(int*)malloc(dim*sizeof(int))))
exit(OVERFLOW);
A.constants[dim-1]=1;
//L=1,指针的增减以元素的大小为单位
for(i=dim-2; i>=0;--i)
A.constants[i]=A.dounds[i+1]*A.constants[i+1];
return OK;  }
```

```
Status DestroyArray(Array&A)  {
  //销毁数组A
 if (!A.dase)  return ERROR;
 free(A.base);   A.base=NULL;
 if(!A.bounds)  return ERROR;
 free(A.bounds);   A.bunds=NULL;
 if(!A.constants)   return ERROR;
 free(A.constants);   A.constants=NULL;
 return OK;   }
```

```
status Locate(Array A, va-list ap, int &off) {
/* 若ap指示的各下标值合法，则求出该元素在A中
   相对址off */
  off=0;
  for(i=0; i<A.dim;++i)   {
      ind =va-arg(ap,int);
      if(ind<0||ind>=A.bounds[i])
            return OVERFLOW;
      off+=A.constants[i]*ind;   }
  return OK;   }
```

```
status Value (Array A, ElemType&e, int
  j1,...)   {
 /* A是n维数组，e为元素变量，随后是n个
   下标值。若各下标不超界，则e赋值为所
   指定的A的元素值，并返回OK */
  va_start(ap,e);
  if((result=Locate(A,ap,off))<=0)
            return result;
  e=*(A.base+off);
  return OK;
}
```

```
status Assign(Array &A,ElemType e, int
  j1 ,...)
{ /*A是n维数组，e为元素变量，随后是n
  个下标值。若下标不超界，则将e的值赋
  给所指定的A的元素，并返回OK。*/
  va_start(ap,e);
  if((result=Locate(A,ap,off))<=0)
          return result;
  *(A.base+off)=e;
  return OK;
}
```

§5.3 矩阵的压缩存储

何谓稀疏矩阵？

假设m 行n 列的矩阵含t 个非零元素，

则称

$$\delta = \frac{t}{m \times n}$$

为稀疏因子。

通常认为δ ≤ 0.05 的矩阵为稀疏矩阵。

以常规方法，即以二维数组表示高阶的稀疏矩阵时产生的问题：

1) 零值元素占了很大空间；

2) 计算中进行了很多和零值的运算，遇除法，还需判别除数是否为零。

解决问题的原则：

1) 尽可能少存或不存零值元素；

2) 尽可能减少没有实际意义的运算；

3) 操作方便。即：

能尽可能快地找到与下标值（i，j）对应的元素，

能尽可能快地找到同一行或同一列的非零值元。

有两类稀疏矩阵：

1) 特殊矩阵

非零元在矩阵中的分布有一定规则

例如：对称矩阵

三角矩阵

2) 随机稀疏矩阵

非零元在矩阵中随机出现

对称矩阵的压缩存储

- 若n阶矩阵A中的元素满足

$$a_{ij}=a_{ji}\ (1\le i,j\le n)$$

则称为n阶对称矩阵。

- 对于n阶对称矩阵可以为每一对对称元分配一个存储空间，则可以将n^2个元压缩存储到n(n+1)/2个元的空间中。
- 以一维数组sa[n(n+1)/2]作为n阶对称矩阵的存储结构，则sa[k]和矩阵元a_{ij}之间存在一一对应的关系：

$$k=\begin{cases} i(i-1)/2+j-1 & \text{当 } i \geq j \\ j(j-1)/2+i-1 & \text{当 } i < j \end{cases}$$

$$\begin{pmatrix} a_{11} & a_{12} & & a_{1n} \\ a_{21} & a_{22} & & a_{2n} \\ & & & \\ & & & \\ a_{m1} & a_{m2} & & a_{mn} \end{pmatrix}$$

0	1	2	3	4	5					n(n+1)/2-1
a_{11}	a_{21}	a_{22}	a_{31}	a_{32}	a_{33}	……	a_{m1}	a_{m2}	……	a_{mn}

- 这种压缩存储的方法同样适用于三角矩阵。所谓下（上）三角矩阵是指矩阵的上（下）三角（不包括对角线）中的元素均为常数c或零的n阶矩阵。
- 则三角矩阵的压缩存储除了和对称矩阵一样只存储其下（上）三角中的元之外，再加一个存储常数c的存储空间即可。

对角矩阵的压缩存储

- 在对角矩阵中所有的非零元素都 集中在以主对角线为中心的带状区域中，即除了主对角线上和直接在对角线上下方若干条对角线上的元之外，所有其他的元都为零。
- 对于对角矩阵可以按照某个规则（或以行为顺序，或以对角线的顺序）将其压缩存储到一维数组中。

例. 三对角矩阵的压缩存储

$$B[n, n] = \begin{pmatrix} b_{11} & b_{12} & & & & \\ b_{21} & b_{22} & b_{23} & & & \\ & b_{32} & b_{33} & b_{34} & & \\ & & \ddots & \ddots & \ddots & \\ & & & \ddots & \ddots & b_{n-1n} \\ & & & & b_{nn-1} & b_{nn} \end{pmatrix}$$

0元素

0元素

非零元素的个数为3n-2

随机稀疏矩阵的压缩存储方法

- 三元组顺序表
- 行逻辑链接的顺序表
- 十字链表

抽象数据类型稀疏矩阵的定义

ADT SparseMatrix {

数据对象：

$D=\{ a_{ij} | i=1,2,\ldots,m;\ j=1,2,\ldots,n;$

$a_{ij} \in ElemSet,$

m和n分别称为矩阵的行数和列数 }

数据关系：　$R=\{ Row, Col \}$

$Row=\{ <a_{ij}, a_{i,j+1}> | 1 \le i \le m,\ 1 \le j \le n-1 \}$

$Col =\{ <a_{ij}, a_{i+1,j}> | 1 \le i \le m-1,\ 1 \le j \le n \}$

- CreateSMatrix(&M);
  操作结果：创建稀疏矩阵M
- DestroySMatrix(&M);
  初始条件：稀疏矩阵M存在
  操作结果：销毁稀疏矩阵M
- PrintSMatrix(M);
  初始条件：稀疏矩阵M存在
  操作结果：输出稀疏矩阵M

- CopySMatrix(&M, &T);
  初始条件：稀疏矩阵M存在
  操作结果：由稀疏矩阵M复制得到T
- MultSMatrix(M, N, &Q);
  初始条件：稀疏矩阵M的列数等于N的行数
  操作结果：求稀疏矩阵乘积Q=M×N
- TransposeSMatrix(M, &T);
  初始条件：稀疏矩阵M存在
  操作结果：求稀疏矩阵M的转置矩阵T

- AddSMatrix(M, N, &Q);

  初始条件：稀疏矩阵M与N的行数和列数对应相等

  操作结果：求稀疏矩阵的和Q=M+N

- SubSMatrix(M, N, &Q);

  初始条件：稀疏矩阵M与N的行数和列数对应相等

  操作结果：求稀疏矩阵的差Q=M-N

} ADT SparseMatrix

稀疏矩阵的压缩存储

- 按照压缩存储的概念，只存储稀疏矩阵中的非零元。因此，除了存储非零元的值之外，还必须同时记录下非零元所在行和列的位置（i，j）。
- 反之，一个三元组（i，j, a_{ij}）唯一确定了矩阵A的一个非零元。
- 由此，稀疏矩阵可以由表示非零元的三元组及其行列数唯一确定。

1. 三元组顺序表

三元组：（i, j, value）

$$\begin{pmatrix} 15 & 0 & 0 & 22 & 0 & -15 \\ 0 & 11 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 91 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 28 & 0 & 0 & 0 \end{pmatrix}$$

例如 ：

（1, 1, 15）表示第1行、第1列、值为15的元素。

（1, 4, 22）表示第1行、第4列、值为22的元素。

（1, 6, -15）表示第1行、第6列、值为-15的元素。

稀疏矩阵的三元组顺序表存储表示

```
#define MAXSIZE 12500
typedef struct {
    int i, j;   //该非零元的行下标和列下标
    ElemType e;    // 该非零元的值
} Triple; // 三元组类型
typedef struct {
    Triple data[MAXSIZE + 1]; //data[0]未用
    int  mu, nu, tu ; //行数、列数和非零元个数
} TSMatrix; // 稀疏矩阵类型
```

$$M = \begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{pmatrix}$$

M.data

	i	j	e
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	6	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

M.mu	6
M.nu	7
M.tu	8

稀疏矩阵的转置运算

- 一个稀疏矩阵的转置矩阵仍然是稀疏矩阵。
- 求转置矩阵的三个步骤：

  ① 将矩阵的行列值互相交换

  ② 将每个三元组中的i和j相互调换

  ③ 重排三元组之间的次序
- 关键是如何处理第三步？

方法一

- 设b为稀疏矩阵a的转置矩阵
- 按照b.data中三元组的次序依次在a.data中找到相应的三元组进行转置，即按照矩阵M的列顺序进行转置。
- 为了找到M的每一列中所有的非零元素，需要对其三元组表a.data从第一行起整个扫描一遍，由于a.data是以M的行序存储非零元的，由此得到的恰是b.data应有的顺序。

```
Status TransposeSMatrix(TSMatrix M,
  TSMatrix &T)  {
   T.mu=M.nu ;   T.nu=M.mu ;   T.tu=M.tu ;
   if(T.tu)  {
     q=1;
     for(col=1;col<=M.nu;++col)
      for(p=1;p<=M.tu;++p)
       if(M.data[p].j==col)  {
         T.data[q].i=M.data[p].j;
         T.data[q].j=M.data[p].i;
         T.data[q].e=M.data[p].e;   ++q;  }  }
   return OK;  }
```

时间复杂度分析

- 转置运算算法1的时间复杂度为 O(nu×tu)
- 非零元的个数tu和矩阵元素个数 mu×nu 同数量级时，转置运算算法1的时间复杂度为

  O(mu×nu×nu)

方法二

- 按照a.data中三元组的次序进行转置，将转置后的三元组置入b中恰当的位置。
- 如果能预先确定矩阵M中每一列（即T中的每一行）的第一个非零元在b.data中应有的位置，则对a.data中的三元组依次做转置时便可以直接放入b.data中恰当的位置。
- 为了确定这些位置，在转置前应先求得M的每一列中非零元素的个数。

- 设num[col]表示矩阵M中第col列中非零元的个数。
- 设cpot[col]表示M中第col列的第一个非零 元在b.data中的位置。
- 则有

cpot[1]=1;

cpot[col]=cpot[col-1]+num[col-1]

$2 \le col \le a.nu$

快速转置算法

```
Status TransposeSMatrix(TSMatrix M,
  TSMatrix &T) {
   T.mu=M.nu ;  T.nu=M.mu ;  T.tu=M.tu ;
   if(T.tu) {
     for(col=1;col<=M.n;++col) num[col]=0;
     for(t=1;t<=M.tu;++t)
         ++num[M.data[t].j];//求M中每一列含非
                                  //零元的个数
     cpot[1]=1;
```

```
// 求第col列中第一个非零元在b.data中的序号
for(col=2;col<=M.nu;++col)
  cpot[col]=cpot[col-1]+num[col-1];
for(p=1;p<=M.tu;++p)  {
  col=M.data[p].j;   q=cpot[col];
  T.data[q].i=M.data[p].j;
  T.data[q].j=M.data[p].i;
  T.data[q].e=M.data[p].e;
  ++cpot[col];  }
}
reutrn OK;   }
```

时间复杂度分析

- 算法中有四个并列的单循环，循环次数分别为mu、tu、tu和tu，其时间复杂度为 $O(mu+tu)$
- 非零元的个数tu和矩阵元素个数mu×nu 同数量级时，算法的时间复杂度为 $O(mu \times nu)$

2. 行逻辑链接的顺序表

- 为了便于随机存取任意一行的非零元，则需知道每一行的第一个非零元在三元组表中的 位置。
- 可以将快速转置矩阵算法中的指示行信息的辅助数组cpot固定在稀疏矩阵的存储结构中。
- 称这种“带行链接信息”的三元组表为行逻辑链接的顺序表。

行逻辑链接的顺序表类型描述

```
typedef  struct  {
   Triple  data[MAXSIZE+1] ;
   int       rpos[MAXRC+1] ;
   int       mu , nu , tu ;
}  RLSMatrix ;
```

- 设有 $Q=M\times N$

其中M是$m_1\times n_1$矩阵，N是$m_2\times n_2$矩阵，且 $n_1 = m_2$

- 则有

$$Q(i,j)=\sum_{k=1}^{n_1} M(i,k)\times N(k,j)$$

- 在求Q中非零元素值的时候，只需对M.data[1..M.tu]中的每个元素(i,k,M(i,k))找到N.data中所有相应的元素(k,j,N(k,j))相乘即可。

- 在行逻辑链接顺序表中N.rpos提供了N中第k行所有非零元素的信息。
- 稀疏矩阵相乘的基本操作：

对于M中每个元素M.data[p]，找到N中所有满足条件

M.data[p].j=N.data[q].i

的元素，求得M.data[p].e和N.data[q].e的乘积，而Q[i][j]是满足条件的乘积的累加和。

稀疏矩阵相乘算法描述

```
Q初始化;
if(Q是非零矩阵) {
  for(arow=1;arow<=M.mu;++arow) {
     // 处理M的每一行
    ctemp[ ]=0;   // 累加器清零
    计算Q中第arow行的积并存入ctemp[]中;
    将ctemp[]中非零元素压缩存储到Q.data;
  } // for
} // if
```

稀疏矩阵相乘算法

```
Status MultSMatrix ( RLSMatrix M ,
  RLSMatrix N, RLSMatrix &Q ) {
 if( M.nu!=N.mu ) return ERROR ;
 Q.mu=M.mu ; Q.nu=N.nu ; Q.tu=0 ;
 if(M.tu*n.tu!=0) { // Q是非零矩阵
  for(arow=1;arow<=M.mu;++arow) {
   ctemp[ ]=0 ; // 当前行各元素累加器清零
   Q.rpos[arow] = Q.tu+1 ;
   if( arow<M.mu ) tp=M.rpos[arow+1] ;
   else tp=M.tu+1 ;
```

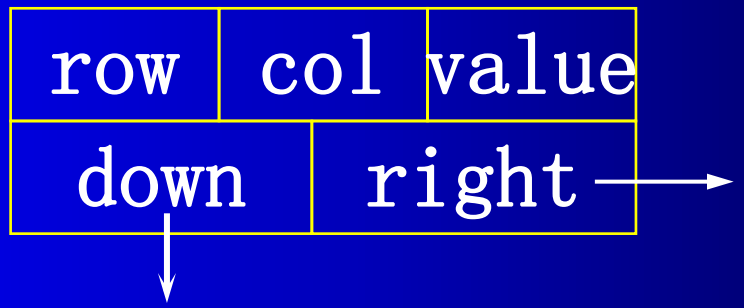
```
for(p=M.rpos[arow];p<tp;++p)//对当前行中每一个元素
{  brow=M.data[p].j;
   if(brow<N.mu)   t=N.rpos[brow+1];
   else   t=n.tu+1;
   for(q=N.rpos[brow];q<t;++q)  {
     ccol=N.data[q].j;  // 乘积元素在Q中的列号
     ctemp[ccol]+=M.data[p].e*N.data[q].e;  }
} // 求得Q中第crow(=arow)行的非零元素
for(ccol=1;ccol<=Q.nu;++ccol) //压缩存储该行非零元素
  if(ctemp[ccol])  {
    if(++Q.tu>MAXSIZE)  return  ERROR ;
    Q.data[Q.tu]=(arow,ccol,ctemp[ccol]) ;  } }
}   return  OK ;   }
```

3. 十字链表

- 为稀疏矩阵的每一行设置一个单独的循环链表，同样为每一列设置一个单独的循环链表。
- 矩阵中每一个非零元素同时包含在两个循环链表中，即包含在它所在的行链表与所在的列链表中，即两个链表的交汇处。

对于一个m×n的稀疏矩阵，分别建立m个行的循环链表与n个列的循环链表，每个非零元素用一个链结点存储。

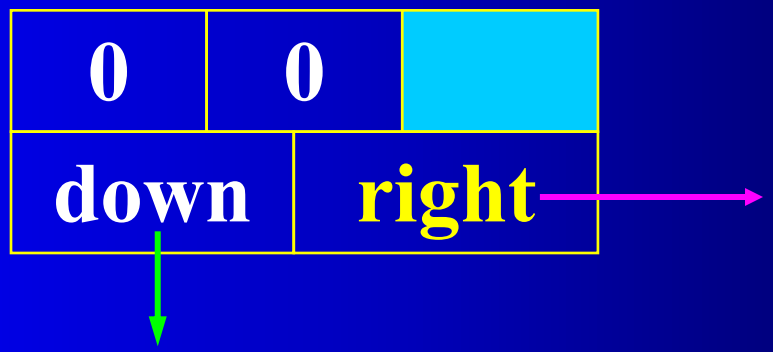
链结点的构造为：

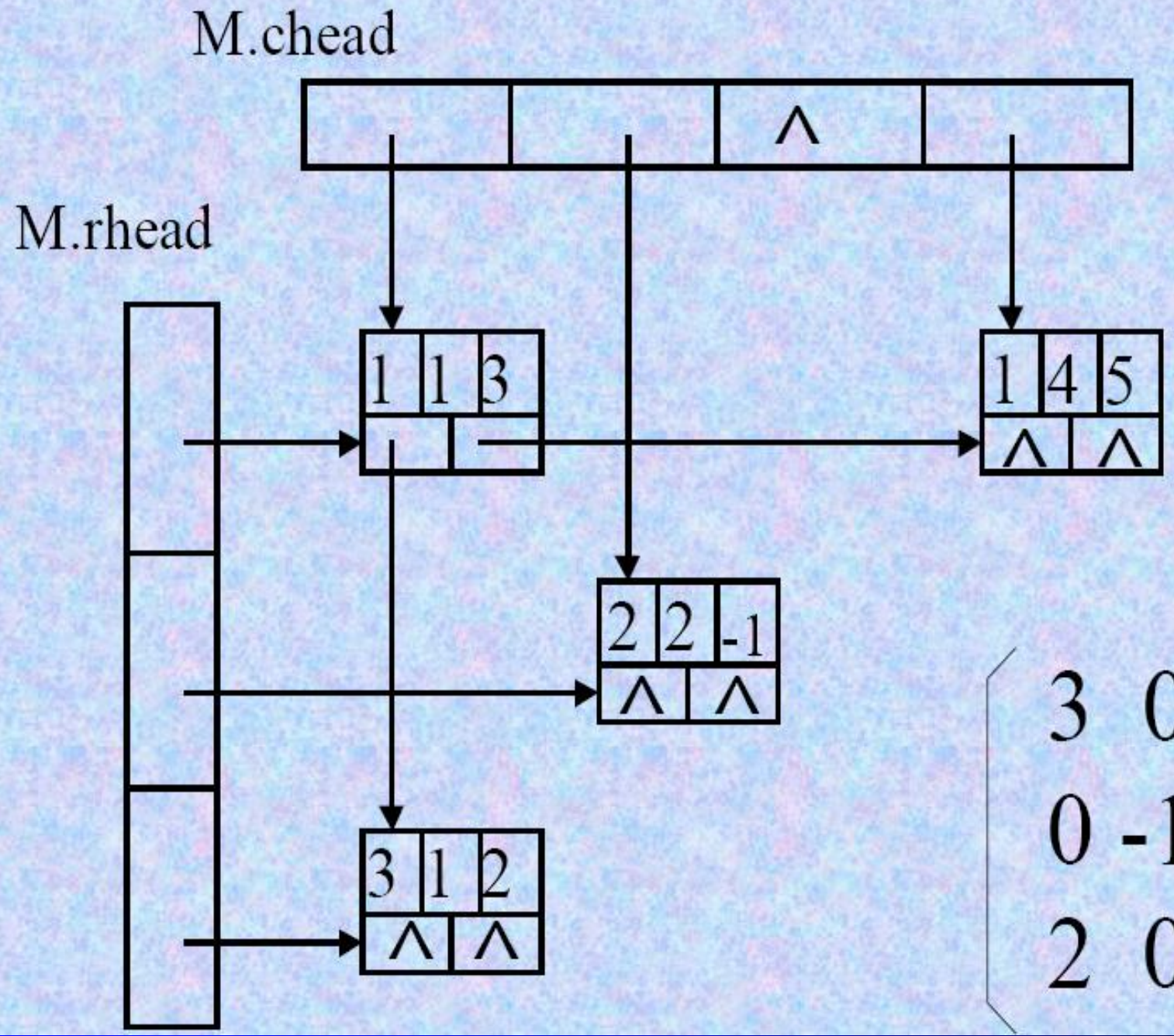


其中，row,col,value分别表示某个非零元素所在的行号、列号和元素的值；down和right分别为向下与向右指针，分别用来链接同一列中与同一行中的所有非零元素对应的链结点。

✓ 对m个行链表，分别设置m个行链表表头结点。表头结点的构造与链表中其他链结点一样，只是令row与col的值分别为0，right域指向相应行链表的第一个结链点。

✓ 对n个列链表，分别设置n个列链表表头结点。 其中，令row和col的值分别为0，down域指向相应列链表的第一个结链点。





$$\begin{pmatrix} 3 & 0 & 0 & 5 \\ 0 & -1 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{pmatrix}$$

稀疏矩阵的十字链表存储表示

```
typedef struct OLNode {
    int  i,j;        // 非零元素的行和列下标
    ElemType  e ;
    struct OLNode  * right, * down ;
} OLNode ,  * OLink ;
typedef struct {
    OLink  * rhead, * chead ;
    int  mu,nu,tu;//稀疏矩阵的行数、列数和非零元素个数
} CrossList ;
```

创建稀疏矩阵的算法

```
Status CreateSMatrix(CrossList  &M) {
  if(M) free(M);
  scanf(&m,&n,&t); //输入M的行数列数非零元素个数
  M.mu=m ;  M.nu=n ;  M.tu=t ;
  M.rhead=(OLink*)malloc((m+1)*sizeof(OLink)));
  M.chead=(OLink*)malloc((n+1)*sizeof(OLink)));
  M.rhead[ ]=M.chead[ ]=NULL ;
  for(scanf(&i,&j,&e);i!=0;scanf(&i,&j,&e)) {
      // 按任意次序输入非零元素
    p=(OLink*)malloc(sizeof(OLink));
    p->i=i;  p->j=j;  p->e=e;
```

```
if(M.rhead[i]==NULL||M.rhead[i]->j>j)
  {p->right=M.rhead[i];M.rhead[i]=p;}
else {   //寻找在行表中的插入位置
   for(q=M.rhead[i]; (q->right)&&q->right->j<j;)
      q=q->right ;
   p->right=q->right ;  q->right=p;   }//完成行插入
if(M.rhead[j]==NULL||M.rhead[j]->i>i)
  {p->down=M.chead[i];M.chead[i]=p;}
else {   //寻找在列表中的插入位置
   for(q=M.chead[i]; (q->down)&&q->down->i<i;)
      q=q->down ;
   p->down=q->down; q->down=p;  }//完成列插入
}  return  OK;   }
```

时间复杂度分析

- 对于m行n列且有t个非零元素的稀疏矩阵，算法的时间复杂度为 $O(t \times s)$， $s=\max\{m,n\}$
- 若按以行序为主序的次序依次输入三元组，则算法的时间复杂度为 $O(t)$

以十字链表表示稀疏矩阵，实现“将矩阵B加到矩阵A上”

- 设指针pa和pb分别指向矩阵A和B中行值相同的两个结点，则矩阵A加矩阵B分为四种情况：
- 若pa==NULL或pa->j>pb->j，则需要在A矩阵的链表中插入一个值为b_{ij}的结点，并改变同一行中前一结点的right域值，及同一列中前一结点的down域值。
- 若pa->j<pb->j，则只要将pa指针向后移动一个结点。

- 若pa->j==pb->j且pa->e+pb->e!=0，则只要将$a_{ij}+b_{ij}$的值赋给pa所指结点的e域即可，其他所有域的值不变。
- 若pa->j==pb->j且pa->e+pb->e==0，则需要在A矩阵的链表值删除pa所指结点，并改变同一行中前一结点的right域值，及同一列中前一结点的down域值。

设指针pre指向pa所指结点在A的行链表上的前驱结点。

在A的每一列的链表上设指针hl[j]，初值为hl[j]=chead[j]

将矩阵B加到矩阵A上的操作过程：

（1）初始化，令pa和pb分别指向A和B的第一行的第一个非零元素结点

```
pa=A.rhead[1];
pb=B.rhead[1];
pre=NULL;
```

(2)重复本步骤，依次处理本行结点，直到B的本行中无非零结点，即pb==NULL为止：

① 若pa==NULL或pa->j>pb->j，则在A中插入一个pb所指结点的复制结点p，A的行表中指针做如下变化：

```
if(pre==NULL)  A.rhead[p->i]=p;
else  { pre->right=p; }
p->right=pa ;    pre=p ;
```

列链表变化首先从hl[p->j]开始找到新结点在同一列中的前驱结点，并使hl[p->j]指向它，然后插入新的结点：

```
if(!A.chead[p->j]||A.chead[p->j]->i>p->i)
{  p->down=A.chead[p->j];
   A.chead[p->j]=p;     }
else  { p->down=hl[p->j]->down;
          hl[p->j]->down=p; }
hl[p->j]=p;
```

②若pa!=NULL且pa->j<pb->j，则令pa指向本行下一个非零元结点，即：

```
pre=pa;
pa=pa->right;
```

③若pa->j==pb->j，则将B中当前结点的值加到A中当前结点上，即

pa->e+=pb->e ;

此时若pa->e!=0，则指针不变，否则删除A中该结点，行表指针变为：

```
if(pre==NULL)  A.rhead[pa->i]=pa->right;
else { pre->right=pa->right; }
p=pa;   pa=pa->right;
```

同时为了改变列表中的指针，首先找到同一列中的前驱结点，使hl[pa->j]指向该结点，然后做如下修改：

```
if(A.chead[p->j]==p)
   A.chead[p->j]=hl[p->j]=p->down;
else  {hl[p->j]->down=p->down; }
free(p);
```

(3)若本行不是最后一行，则令pa和pb指向下一行的第一个非零元结点，转向（2），否则结束。

时间复杂度分析

- 整个运算过程在于对A和B的十字链表逐行扫描，其循环次数主要取决于A和B矩阵中非零元素的个数ta和tb
- 则此算法的时间复杂度为

$$O(ta+tb)$$

§5.4 广义表的类型定义

ADT Glist {

数据对象：

$D = \{e_i \mid i=1,2,..,n;\ n \geqslant 0;$
$e_i \in AtomSet$ 或 $e_i \in GList,$
AtomSet为某个数据对象集合$\}$

数据关系：

$LR = \{<e_{i-1}, e_i> \mid e_{i-1}, e_i \in D, 2 \leqslant i \leqslant n\}$

基本操作：

结构的创建和销毁

InitGList(&L); DestroyGList(&L);

CreateGList(&L, S); CopyGList(&T, L);

状态函数

GListLength(L); GListDepth(L);

GListEmpty(L); GetHead(L); GetTail(L);

插入和删除操作

InsertFirst_GL(&L, e);

DeleteFirst_GL(&L, &e);

遍历

Traverse_GL(L, Visit());

广义表是递归定义的线性结构，

$$LS = (\alpha_1, \alpha_2, \cdots, \alpha_n)$$

其中：α_i 或为原子 或为广义表

其中，LS为广义表的名字，a_i为表中元素；a_i可以是原子元素，也可以是一个子表。n为表的长度，长度为0的表称为空表。

- ✓ 若a_i 为不可再分割的原子元素，则称a_i 为原子元素；
- ✓ 若a_i 为一个子表，则称a_i 为表元素。

广义表的例子

A=( ) —— 长度为0的空表。

B=(a) —— 长度为1，且只有一个原子元素的广义表

C=(a, (b, c)) —— 长度为2的广义表。

D=(A, B, c) —— 长度为3的广义表。

E=(a, E) —— 长度为2的递归的广义表。

广义表是一个多层次的线性结构

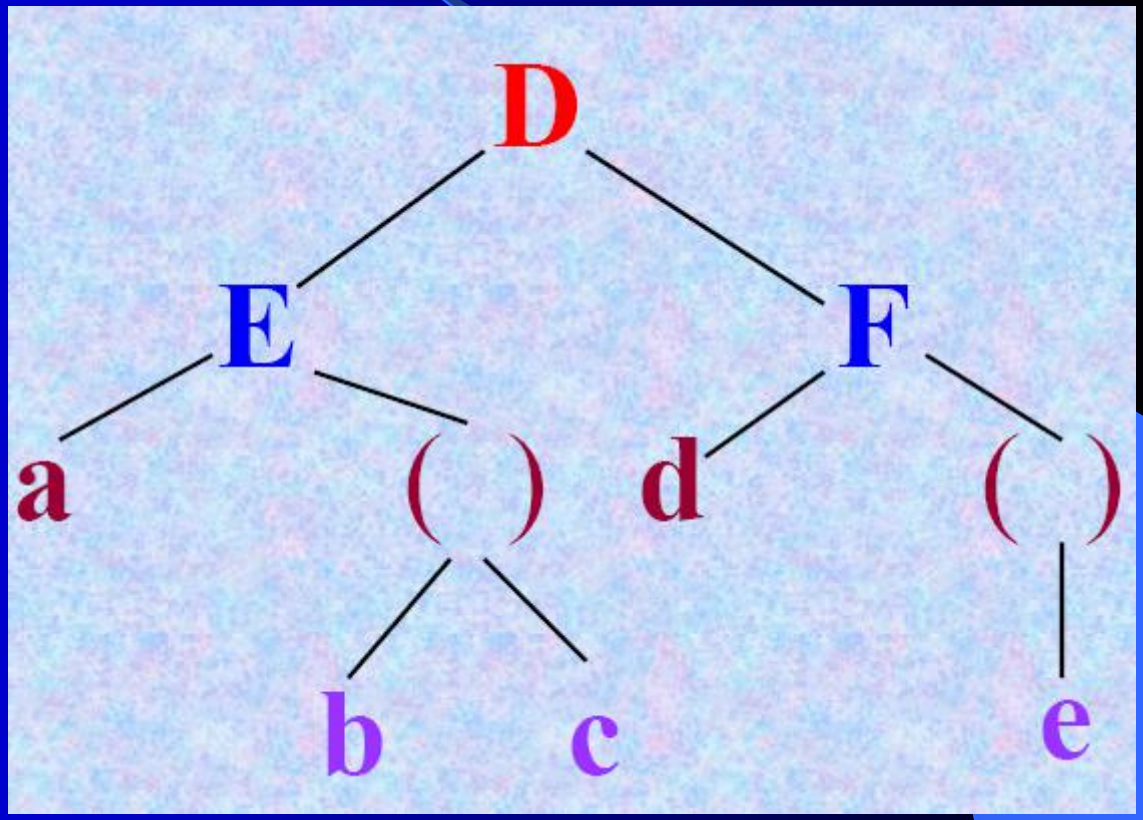
例如:

D=(E, F)

其中:

E=(a, (b, c))

F=(d, (e))



广义表 LS = ($\alpha_1, \alpha_2, \ldots, \alpha_n$)的结构特点:

1) 广义表中的数据元素有相对次序;
2) 广义表的长度定义为最外层包含元素个数
3) 广义表的深度定义为所含括弧的重数;

注意: “原子”的深度为 0

“空表”的深度为 1

4) 广义表可以共享;
5) 广义表可以是一个递归的表。

递归表的深度是无穷值，长度是有限值。

6) 任何一个非空广义表 $LS = (\alpha 1, \alpha 2, \ldots, \alpha n)$ 均可分解为

表头 $Head(LS) = \alpha 1$ 和

例如: $D = (E, F) = ((a, (b, c)), F)$

$Head(\mathbf{D}) = E$ $Tail(\mathbf{D}) = (F)$

$Head(\mathbf{E}) = a$ $Tail(\mathbf{E}) = ((b, c))$

$Head(\mathbf{((b, c))}) = (b, c)$ $Tail(\mathbf{((b, c))}) = ()$

$Head(\mathbf{(b, c)}) = b$ $Tail(\mathbf{(b, c)}) = (c)$

$Head(\mathbf{(c)}) = c$ $Tail(\mathbf{(c)}) = ()$

§5.5 广义表的存储结构

通常采用头、尾指针的链表结构。需要两种结构的结点：一种是表结点，用以表示列表；另一种是原子结点，表示原子。

表结点：

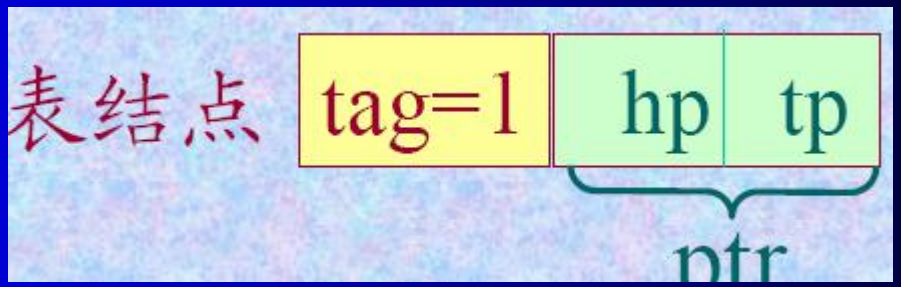
tag=1	hp	tp

原子结点：

tag=0	data

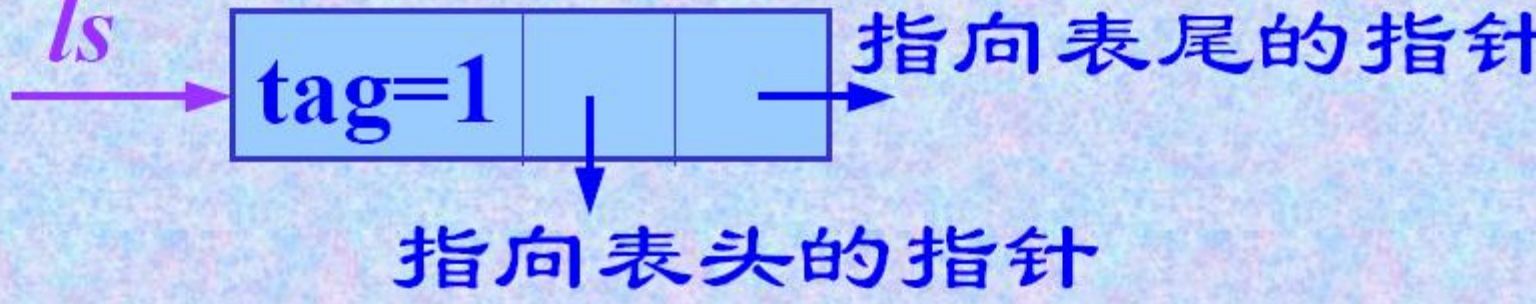
广义表的头尾链表存储表示：

```
typedef enum {ATOM, LIST} ElemTag;
        // ATOM==0:原子, LIST==1:子表
typedef struct GLNode {
   ElemTag    tag;   // 标志域
   union{
      AtomType    atom;   // 原子结点的数据域
      struct  {struct GLNode *hp, *tp;}    ptr;
   };
} *GList
```



空表　*ls=NIL*

非空表 *ls*

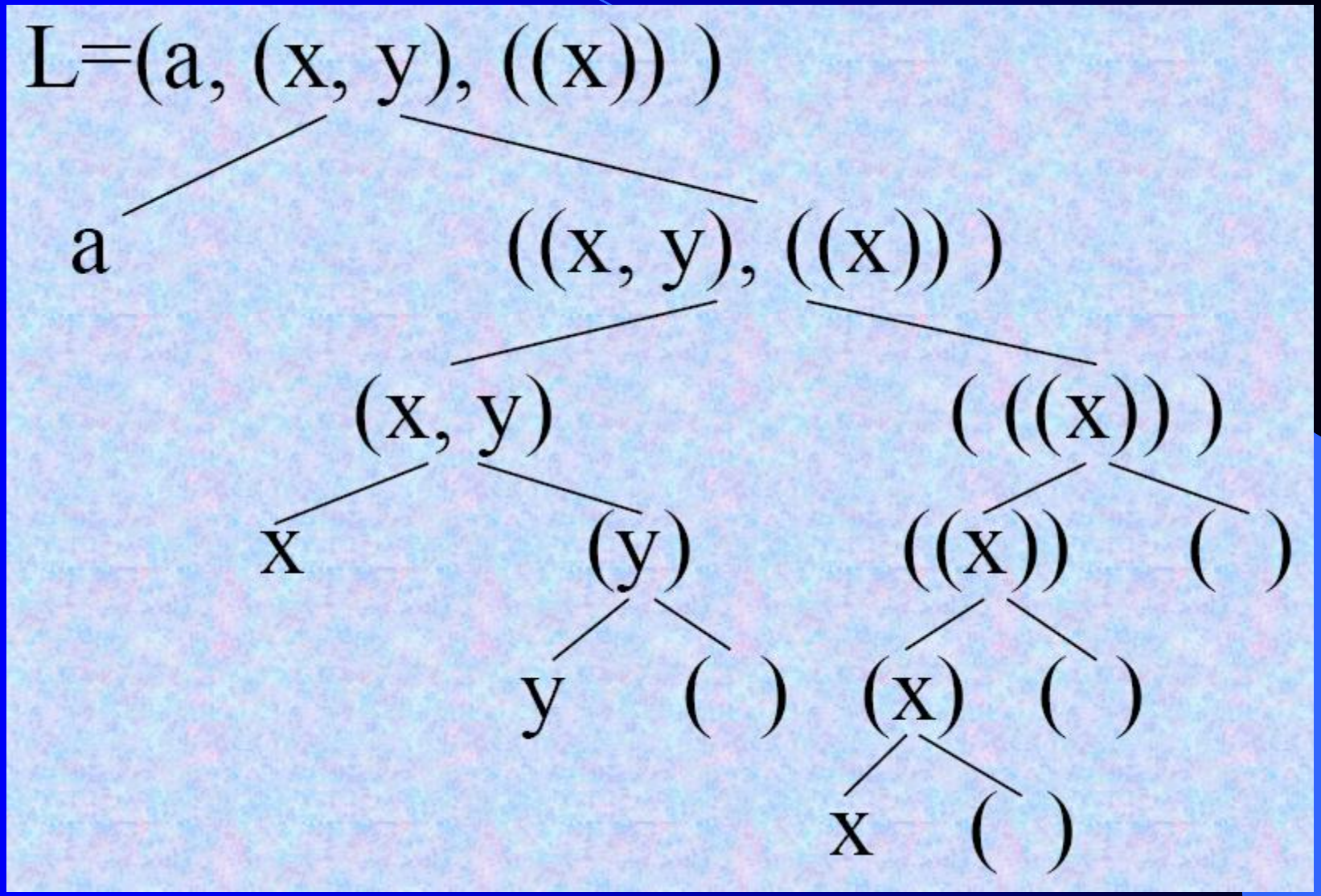


若表头为原子，则为

tag=0	data

否则，依次类推。

例如:

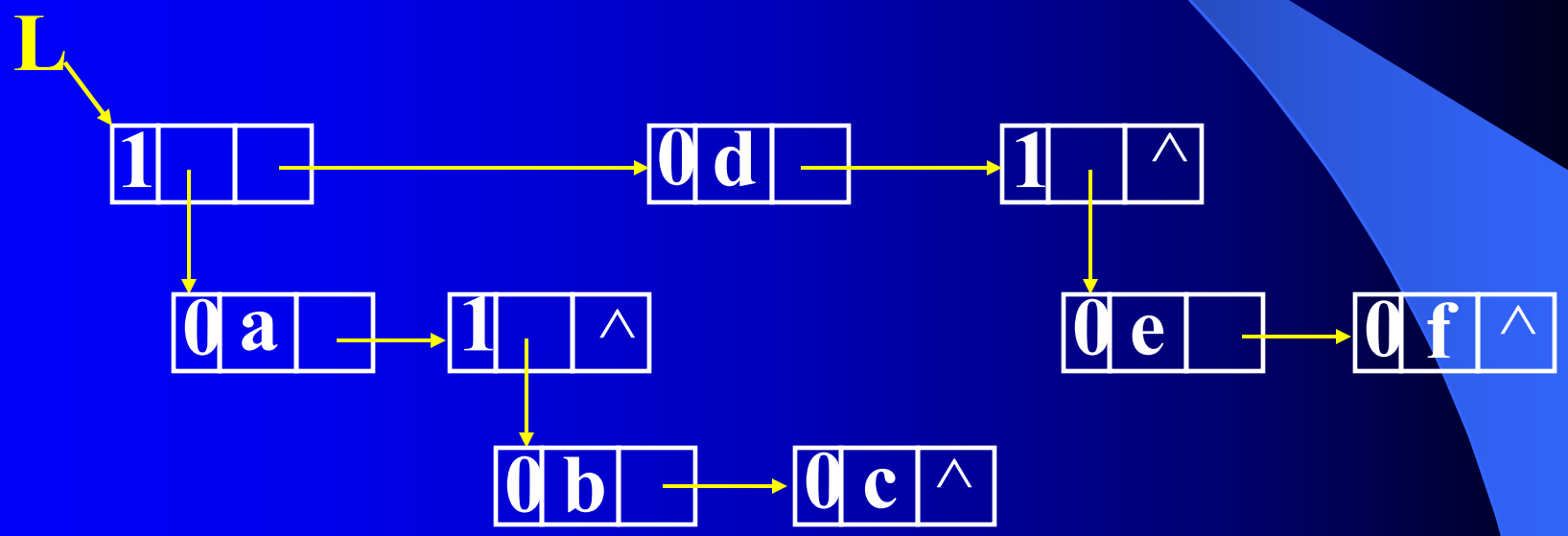


说明：

- 除空表的表头指针为空外，对任何非空列表，其表头指针均指向一个表结点，且该结点中的hp域指示列表表头（或为原子结点，或为表结点），tp域指向列表表尾（除非表尾为空，则指针为空，否则必为表结点）。
- 最高层的表结点的个数即为列表的长度。

例

L=((a, (b,c)), d, (e,f))



比较广义表和线性表的结构特点：

- 相似处：都是链表结构。
- 不同处：

1）广义表的数据元素可能还是个广义表；

2）删除时，不仅要删除原子结点，还需要删除相应的表结点。

1. 了解数组的两种存储表示方法，并掌握数组在以行为主的存储结构中的地址计算方法。

2. 掌握对特殊矩阵进行压缩存储时的下标变换公式。

3. 了解稀疏矩阵的两类压缩存储方法（三元组和十字链表）的特点和适用范围。

4. 掌握广义表的结构特点及其存储表示方法，学会对非空广义表进行分解的分析方法：即可将一个非空广义表分解为表头和表尾两部分。